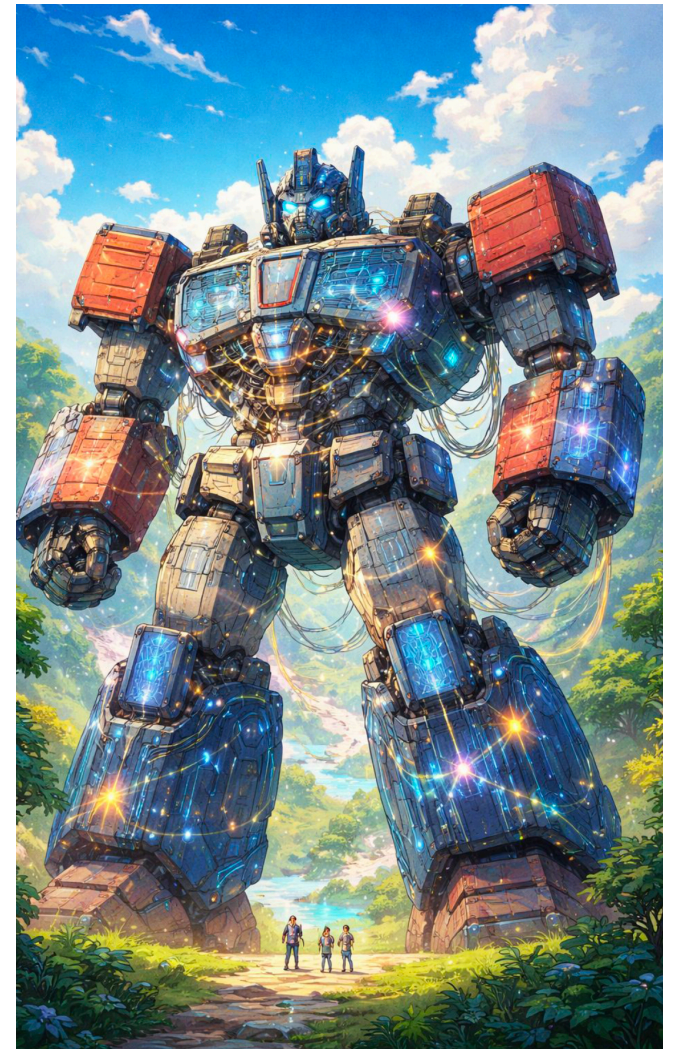


3008ICT Deep Learning

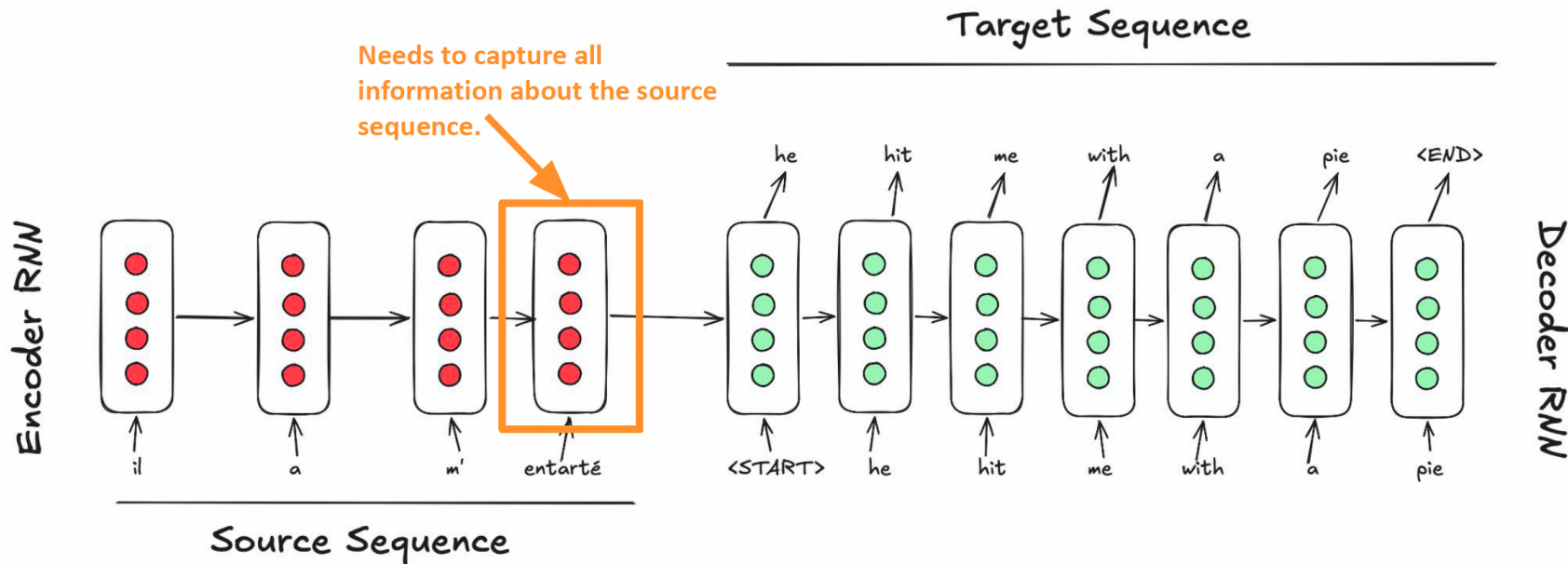
Week 9. Transformer and Attention

Adapted from slides at d2l.ai and Stanford CS224n

Attention Is All You Need Ashish Vaswani et al., 2017

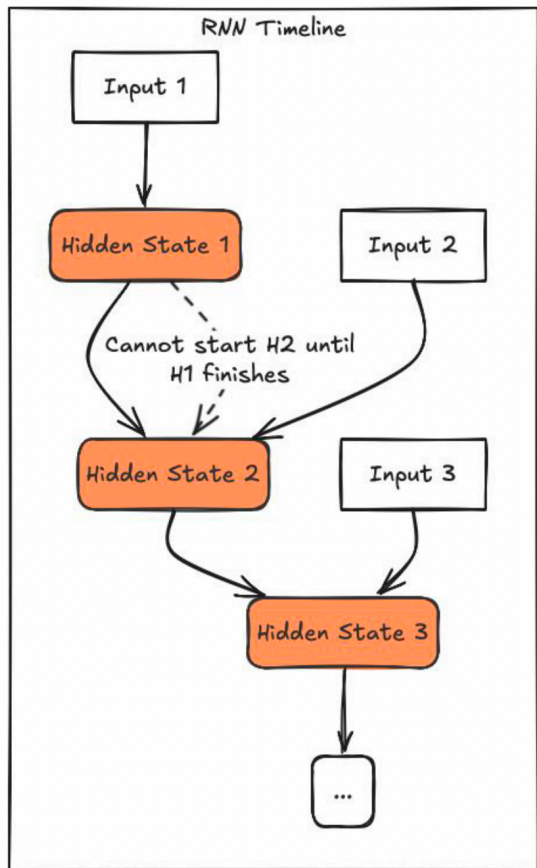


Issues with Recurrent Models



Information bottleneck!

Lack of Parallelizability



The "Hardware Bottleneck" Problem

- **Sequential Dependency:** Step t cannot begin until step $t-1$ is complete.
- **GPU Underutilization:** Modern hardware is built for parallel "grid" work, but RNNs force "single-lane" processing.
- **The Scaling Wall:** Training on massive datasets (Web-scale) becomes computationally impossible due to time constraints.

From Recurrence to Attention



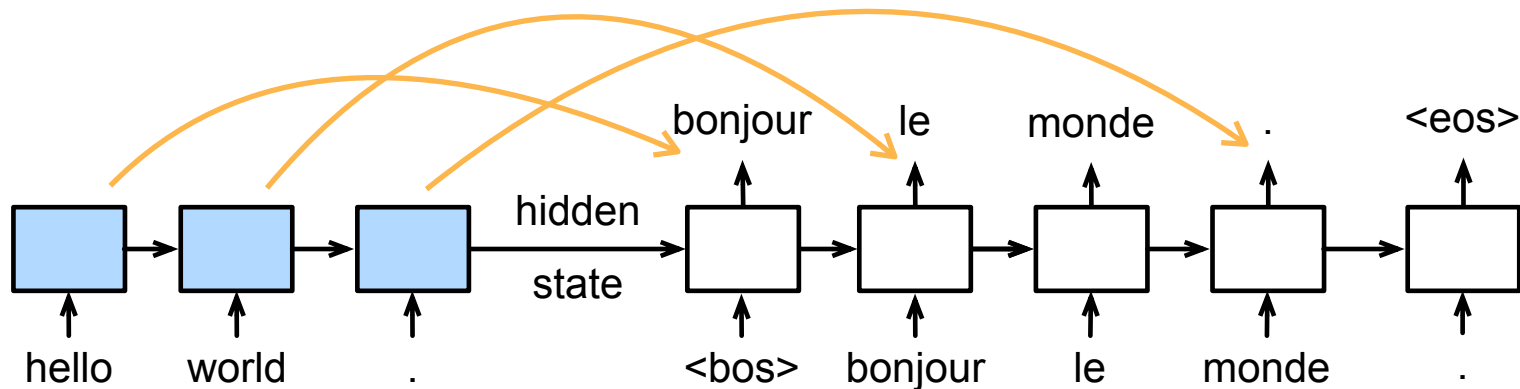
- **$O(1)$ Path Length:** Every token "talks" directly to every other token simultaneously, eliminating the information decay found in RNNs.
- **Sequential Bottleneck Removed:** Computation is no longer restricted by sequence length N , allowing for massive parallelization on modern GPUs.
- **Dynamic Attention Routing:** Replaces rigid "fixed-size" memory vectors with a flexible system that queries the entire sequence for relevant context.

[illegible]

- Attention mechanisms: an integral part of major sequence modeling
 - allowing modeling of dependencies between distant parts of data.
- Attention mechanisms: used in conjunction with a recurrent network.

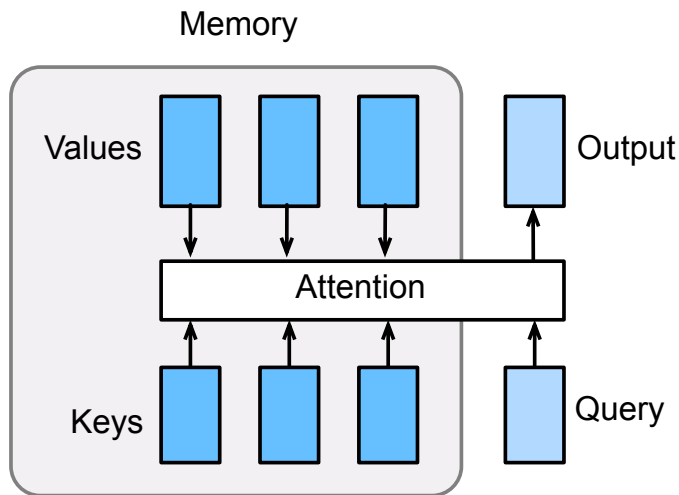
Motivation

- Each generated token might be related to different source tokens



Attention Layer

- An attention layer explicitly select related information
 - Its memory consists of key-value pairs
 - The output will be close to the values whose keys are similar to the query



Attention Layer

- Query $\mathbf{q} \in \mathbb{R}^{d_q}$ and the memory $(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_n, \mathbf{v}_n)$ with $\mathbf{k}_i \in \mathbb{R}^{d_k}$ $\mathbf{v}_i \in \mathbb{R}^{d_v}$
- Compute n scores $a_1, \dots, a_n$ with $a_i = \alpha(\mathbf{q}, \mathbf{k}_i)$
- Use softmax to obtain the attention weights

$$b_1, \dots, b_n = \text{softmax}(a_1, \dots, a_n)$$

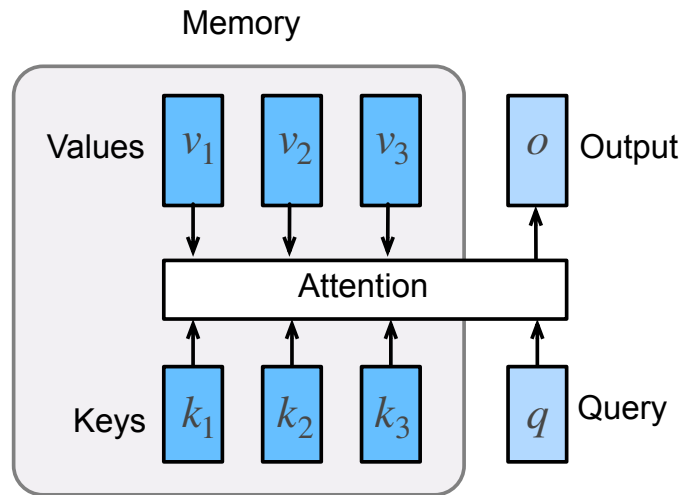
- The output is a weighted sum of values

$$\mathbf{o} = \sum_{i=1}^n b_i \mathbf{v}_i$$

The query $\mathbf{q}$: the context or query

The key $\mathbf{k}_i$: input at time step i .

Vary α to get different attention layers



Dot Product Attention

- Assumption: the query has the same length as values

- All vectors are row vectors

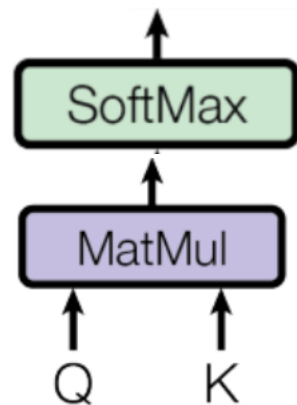
- Given a query $\mathbf{q}$ and a key $\mathbf{k}$,

$$\alpha(\mathbf{q}, \mathbf{k}) = \langle \mathbf{q}, \mathbf{k} \rangle / \sqrt{d}$$

- Vectorised version:

m queries $\mathbf{Q} \in \mathbb{R}^{m \times d}$, n keys $\mathbf{K} \in \mathbb{R}^{n \times d}$

$$\alpha(\mathbf{Q}, \mathbf{K}) = \mathbf{Q}\mathbf{K}^T / \sqrt{d}$$



Implementation of Dot Product Attention (simple)

```
[ ]: import numpy as np
def attention_score(query, key):
    """
    Implements basic attention scoring mechanism
    query: shape (query_len, d_k)
    key: shape (key_len, d_k)
    """
    scores = np.dot(query, key.T)

    d_k = query.shape[-1]
    scaled_scores = scores / np.sqrt(d_k)

    attention_weights = np.exp(scaled_scores) / np.sum(np.exp(scaled_scores), axis=-1, keepdims=True)

    return attention_weights
```

```
[2]: # Example usage
query = np.random.randn(4, 8) # 4 queries, dimension 8
key = np.random.randn(6, 8) # 6 keys, dimension 8
weights = attention_score(query, key)
print("Attention Weights Shape:", weights.shape)
print("Sample Weights:\n", weights[0]) # Weights for first query
```

Attention Weights Shape: (4, 6)

Sample Weights:

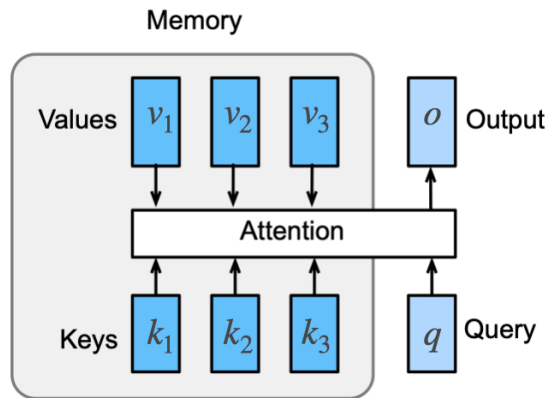
[0.04790467 0.07297749 0.22813667 0.17495425 0.25115171 0.22487522]

Multilayer Perception Attention

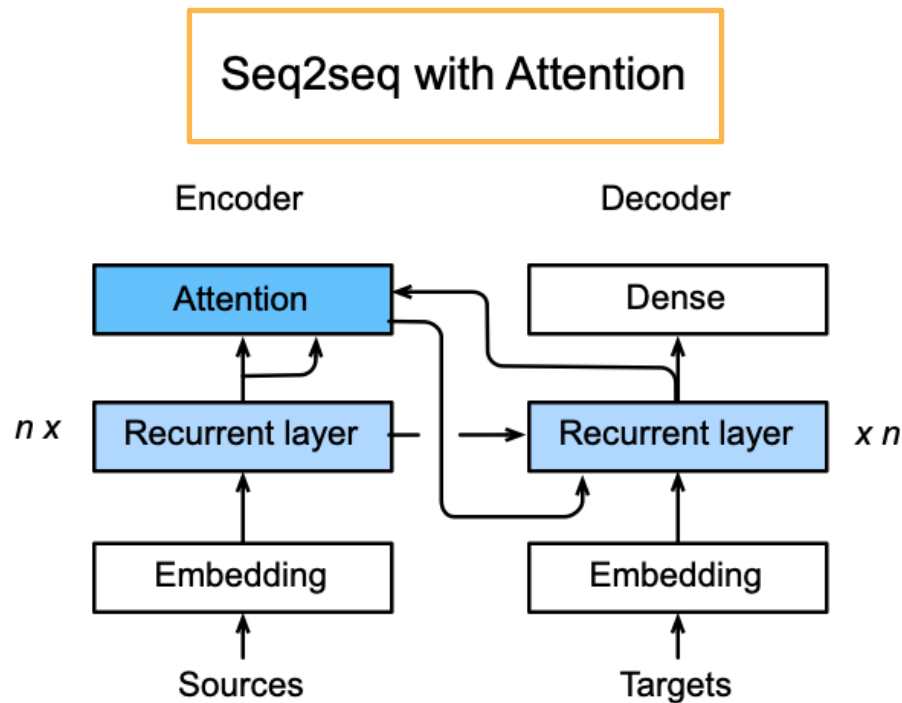
- Learnable parameters $\mathbf{W}_k \in \mathbb{R}^{h \times d_k}$, $\mathbf{W}_q \in \mathbb{R}^{h \times d_q}$, and $\mathbf{v} \in \mathbb{R}^h$

$$\alpha(\mathbf{k}, \mathbf{q}) = \mathbf{v}^T \tanh(\mathbf{W}_k \mathbf{k} + \mathbf{W}_q \mathbf{q})$$

- Equals to concatenate the key and query, and then feed into a single hidden-layer perception with hidden size h and output size 1

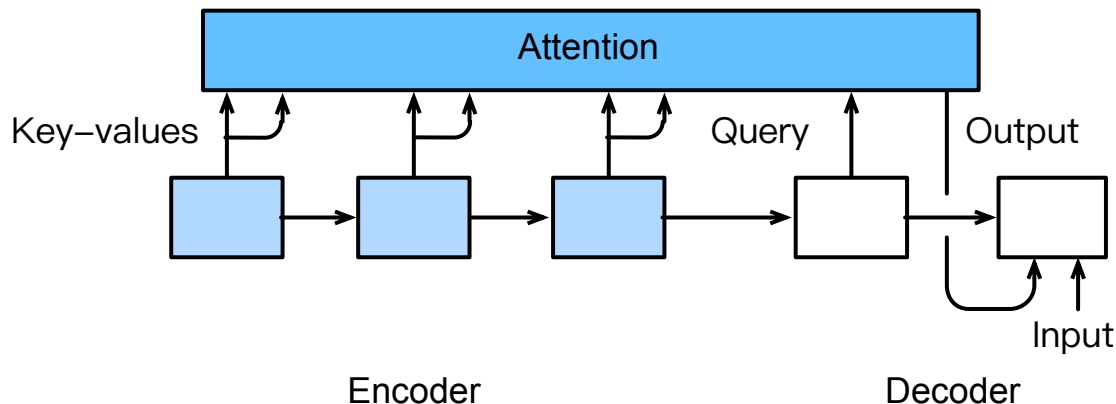


Seq2seq with Attention



Model Architecture

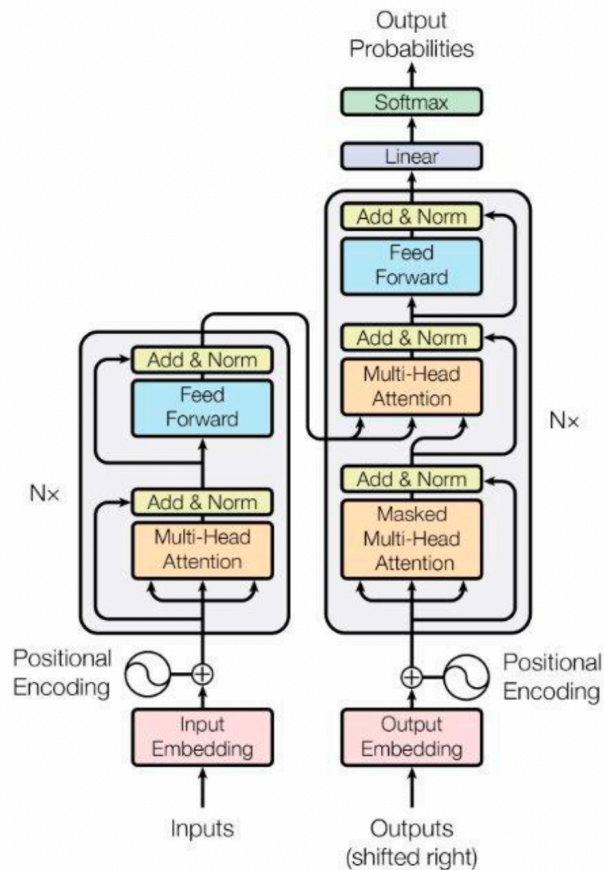
- Add an additional attention layer to use encoder's outputs as memory
- The attention output is used as the decoder's input



Transformer

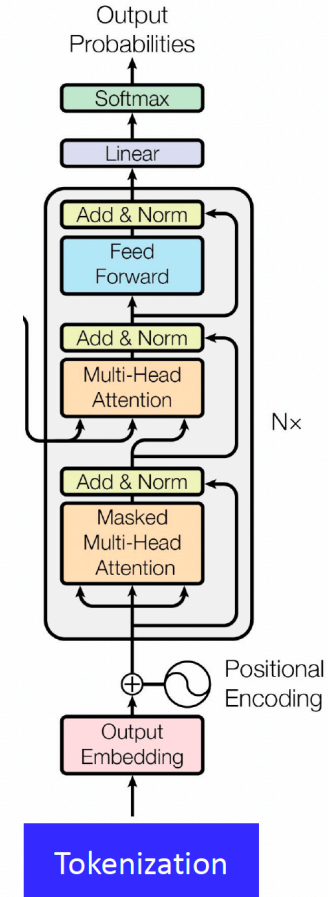


Transformer Architecture



Transformer Architecture

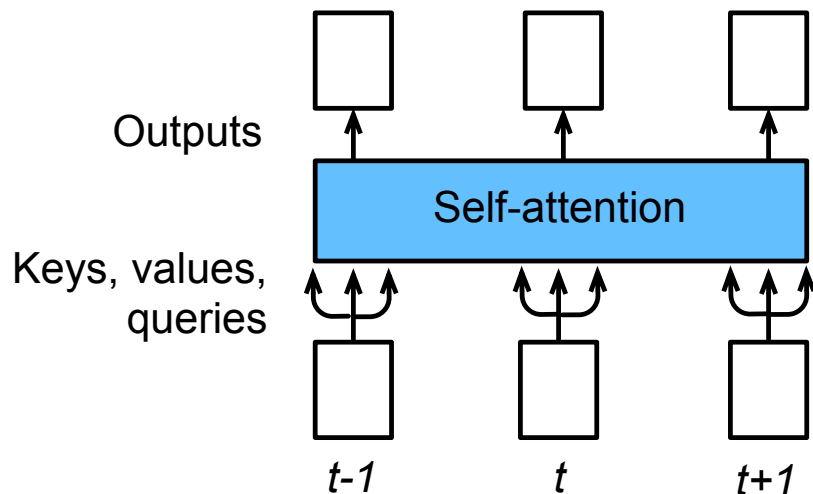
- Tokenization
- Embedding
- Positional Encoding
- (Masked) Multi-Head Attention
- Feed-Forward
- Add & Norm
- Output Projection Layer



**Ref. L19-Transformer.pdf in
2026_3008ICT/Lecture**

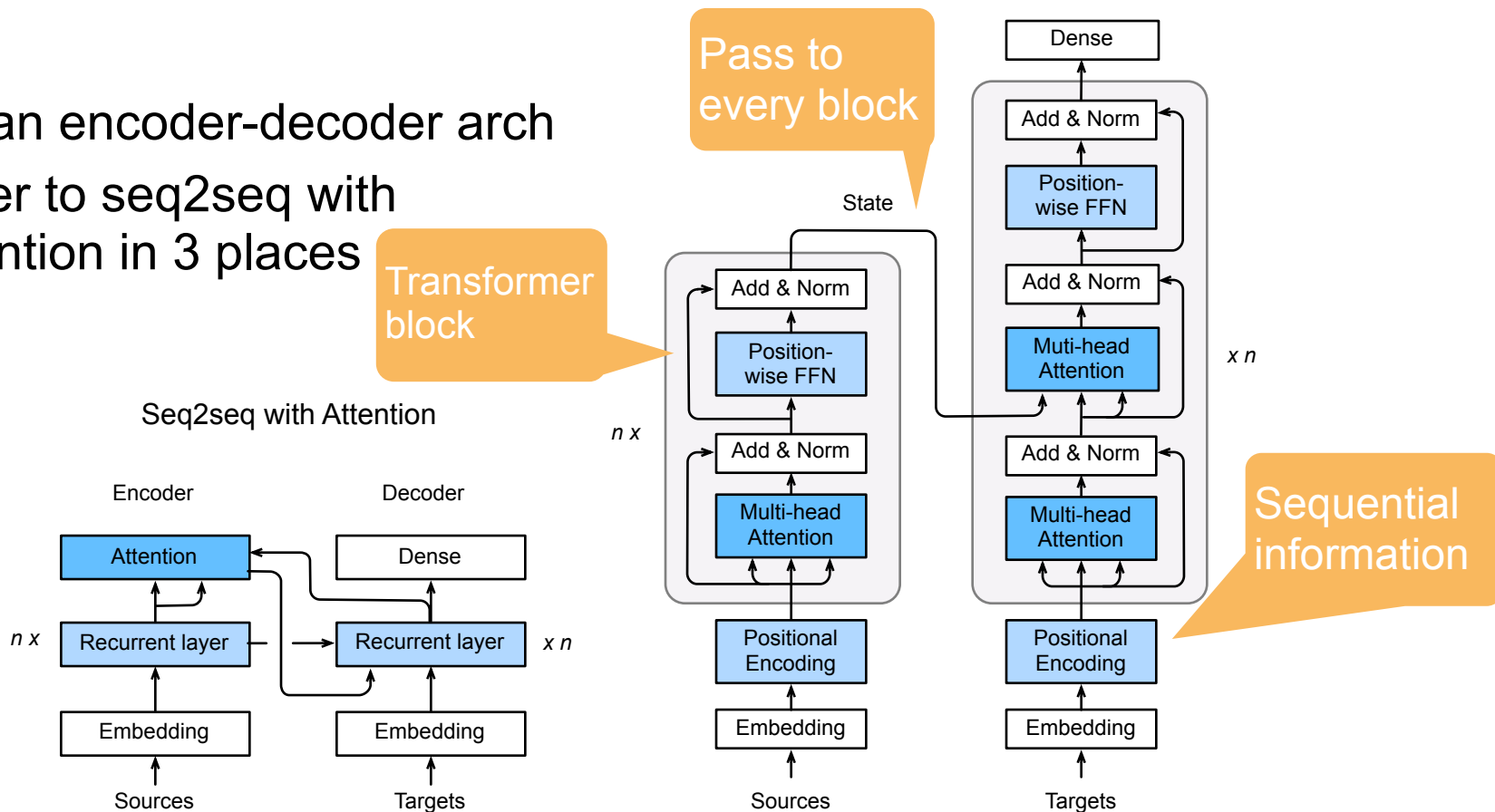
Self-attention

- To generate n outputs with n inputs, we can copy each input into a key, a value and a query
- No sequential information is preserved
- Run in parallel

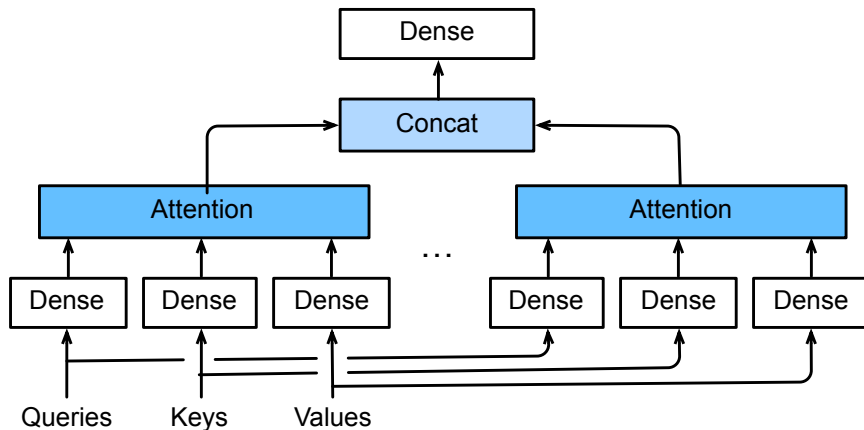


Transformer Architecture

- It's an encoder-decoder arch
- Differ to seq2seq with attention in 3 places



Multi-head Attention

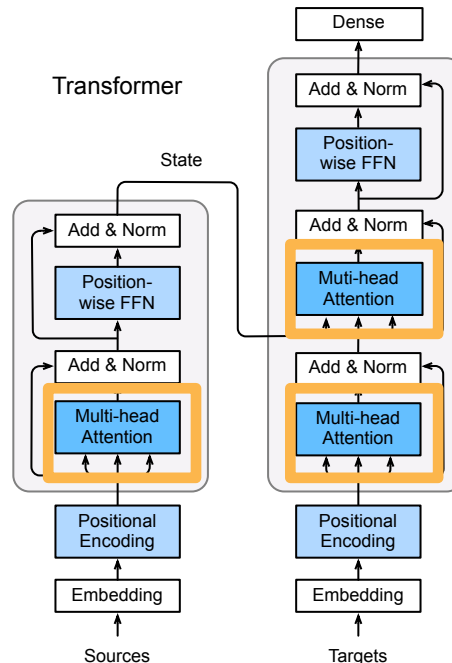


$$\mathbf{W}_q^{(i)} \in \mathbb{R}^{p_q \times d_q}, \mathbf{W}_k^{(i)} \in \mathbb{R}^{p_k \times d_k}, \text{ and } \mathbf{W}_v^{(i)} \in \mathbb{R}^{p_v \times d_v}$$

$$\mathbf{o}^{(i)} = \text{attention}(\mathbf{W}_q^{(i)} \mathbf{q}, \mathbf{W}_k^{(i)} \mathbf{k}, \mathbf{W}_v^{(i)} \mathbf{v})$$

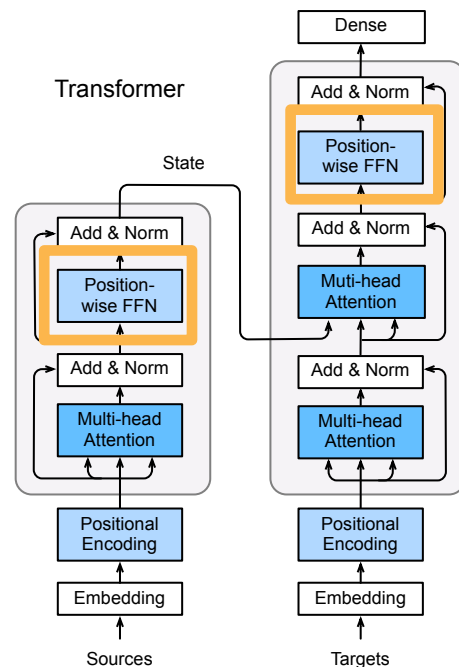
for $i = 1, \dots, h$

$$\mathbf{o} = \mathbf{W}_o \begin{bmatrix} \mathbf{o}^{(1)} \\ \vdots \\ \mathbf{o}^{(h)} \end{bmatrix}$$



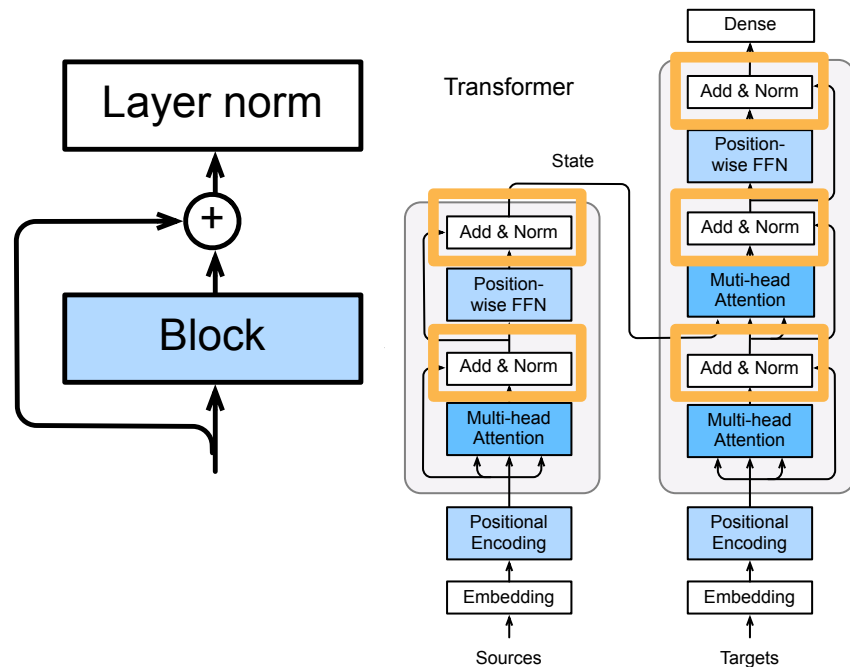
Position-wise Feed-Forward Networks

- Reshape input (batch, seq len, fea size) into (batch * seq len, fea size)
- Apply a two layer MLP
- Reshape back into 3-D
- Equals to apply two (1,1) conv layers



Layer Normalisation: Add and Norm

- Layer norm is similar to batch norm
- But the mean and variances are calculated along the last dimension
- `X.mean(axis=-1)` instead of the first batch dimension in batch
norm `X.mean(axis=0)`



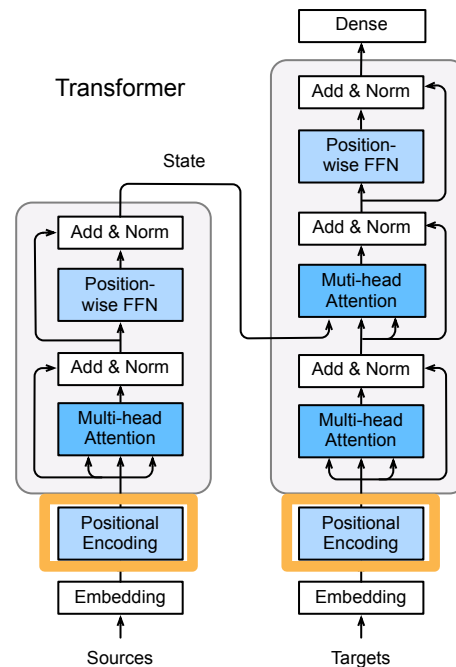
Positional Encoding

- Assume embedding output $X \in \mathbb{R}^{l \times d}$ with shape (seq len, embed dim)
- Create $P \in \mathbb{R}^{l \times d}$ with

$$P_{i,2j} = \sin(i/10000^{2j/d})$$

$$P_{i,2j+1} = \cos(i/10000^{2j/d})$$

- Output $X + P$



Predicting

- Predict at time t :
 - Inputs of previous times as keys and values
 - Input at time t as query, as well as key and value, to predict output

